Universidad Rafael Landívar.
Facultad de Ingeniería.
Licenciatura en Ingeniería en Informática y Sistemas.
Estructura de Datos 2
Docente: Ing. Manuel Rojas

Manual Técnico

“Red de Transporte Urbano”

Estudiante:
Luis Manuel Velásquez González, 1502325
José Roderico Noguera González, 1619125
Moshé Arenz Pelaez Virula, 1556425

Manual Técnico — Red de Transporte Urbano

1. Introducción

Este manual describe el diseño y la implementación de la aplicación **Red de Transporte Urbano**. El sistema administra las paradas y rutas de buses de una ciudad (el conjunto de datos de ejemplo usa lugares de Quetzaltenango) y permite: registrar, buscar y eliminar paradas; sugerir nombres mientras se escribe (autocompletado); recorrer la red con BFS y DFS; calcular la ruta más rápida o la más económica con Dijkstra; y verificar si dos paradas pertenecen a la misma zona conectada.

Tecnología. C# sobre .NET 10 con Windows Forms. La solución `TransporteUrbanoEDD.slnx` tiene tres proyectos:

- `TransporteUrbanoEDD.Core`: modelos, estructuras de datos propias y carga/guardado de CSV. No depende de la interfaz.
- `TransporteUrbanoEDD.UI`: ventana principal y tres pestañas (controles de usuario).
- `Pruebas`: aplicación de consola con pruebas de las estructuras del Core.

Regla de implementación propia. La Tabla Hash, el Grafo, el Union-Find y el Trie se implementaron sin `List<T>`, `Dictionary`, `HashSet`, `SortedSet` ni `PriorityQueue`. Los únicos tipos nativos usados en la lógica son arreglos primitivos, `string`/`StringBuilder` y `Queue<int>`, esta última solo como cola de control del recorrido BFS (permitido por el enunciado). Hasta el montículo de Dijkstra y la lista enlazada son propios.

Datos dinámicos. Las paradas y rutas se cargan desde `paradas.csv` y `rutas.csv` (carpeta `Datos`, copiada junto al ejecutable) o se ingresan manualmente desde la interfaz.

2. Responsables por estructura

Estructura / módulo	Archivo	Responsable	Pestaña de la UI
Tabla Hash	`Estructuras/TablaHashParadas.cs`	Integrante 1 · Arenz Peláez	1. Paradas y Zonas
Union-Find	`Estructuras/UnionFindZonas.cs`	Integrante 1 · Arenz Peláez	1. Paradas y Zonas
Lista enlazada, modelo `Parada`, `CargadorCSV`	`ListaPropia.cs`, `Parada.cs`, `CargadorCSV.cs`	Integrante 1 · Arenz Peláez	1. Paradas y Zonas
Trie (autocompletado)	`Estructuras/TrieParada.cs`	Integrante 2 · José Noguera	2. Buscador
Orquestador (integración)	`Estructuras/OrquestadorTransporte.cs`	Integrantes 1 y 2	2. Buscador

Estructura / módulo	Archivo	Responsable	Pestaña de la UI
Grafo (BFS, DFS, Dijkstra), Min-Heap, `AristaRuta`, `ResultadoRuta`, `NodoHeap`	`GrafoTransporte.cs`, `MinHeapPropio.cs`, `Modelos/*`	Integrante 3 · Luis Velasquez	3. Rutas y Mapa

3. Arquitectura e integración de las estructuras

La aplicación tiene tres capas: **interfaz** (`FormPrincipal` y sus tres pestañas), **orquestación** (`OrquestadorTransporte` y `CargadorCSV`) y **estructuras de datos** (Tabla Hash, Grafo, Union-Find, Trie, lista enlazada y min-heap).

`FormPrincipal` crea **un único** `OrquestadorTransporte`, que es dueño de las cuatro estructuras y del cargador. Las tres pestañas reciben esas mismas instancias, de modo que un cambio hecho en una pestaña se ve en las demás. Cuando la pestaña 1 agrega, elimina o carga parada dispara el evento `DatosModificados`; la ventana principal refresca entonces el buscador, el mapa y la barra de estado.

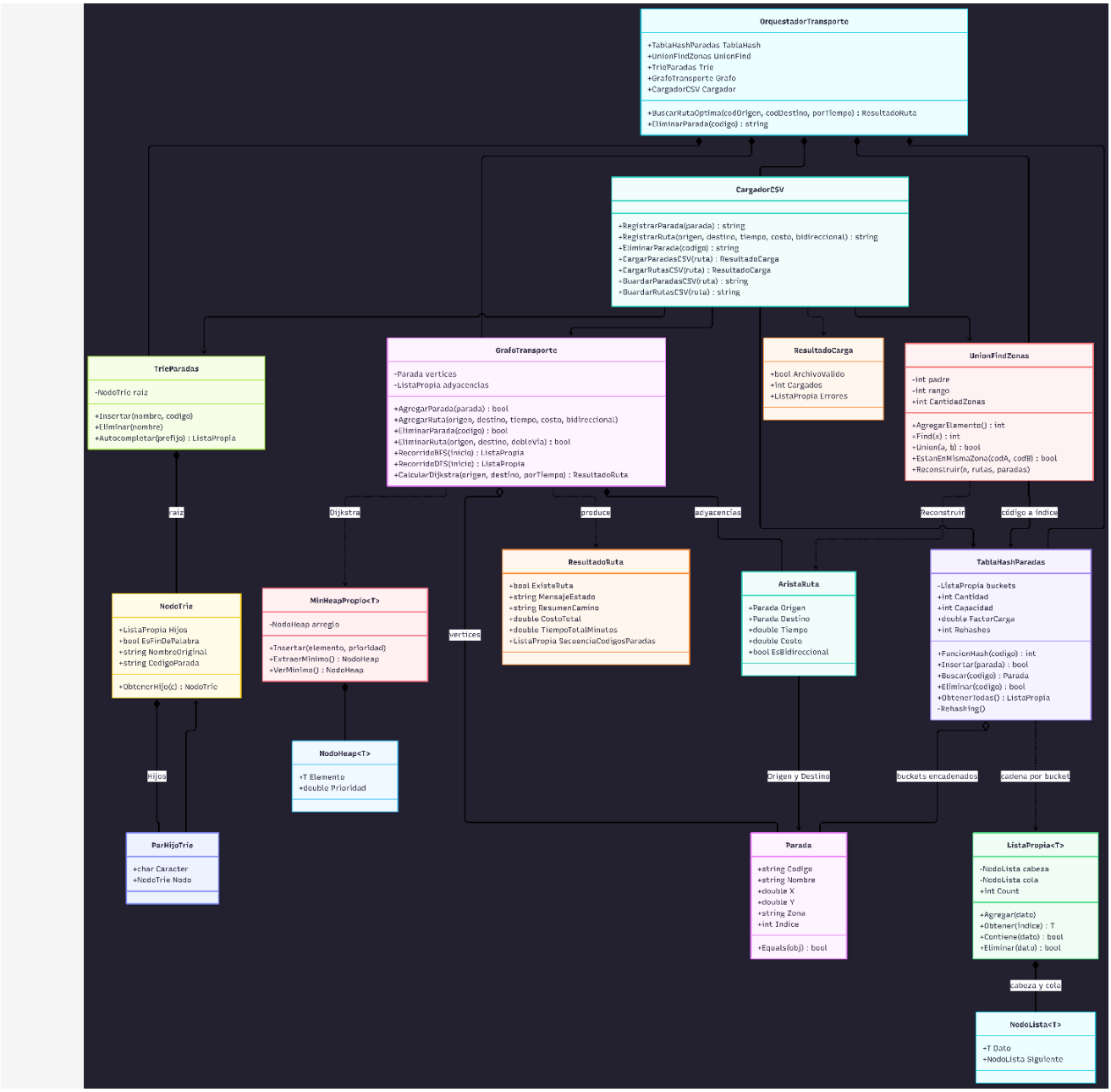
Las cuatro estructuras trabajan coordinadas en cada operación:

Operación	Estructuras que intervienen y en qué orden
Registrar parada	Tabla Hash (valida clave única) → Grafo (crea el vértice y define el índice) → Union-Find (nuevo conjunto con ese índice) → Trie (inserta el nombre)
Registrar ruta (CSV)	Tabla Hash (verifica que existan ambos extremos) → Grafo (crea la arista) → Union-Find (une las dos paradas en una zona)
Calcular ruta por nombre	Trie (sugiere el nombre) → Tabla Hash (devuelve la parada) → Union-Find (valida la zona) → Grafo + Min-Heap (Dijkstra)
Eliminar parada	Tabla Hash (baja) → Grafo (baja del vértice y purga de aristas) → Union-Find (reconstrucción)

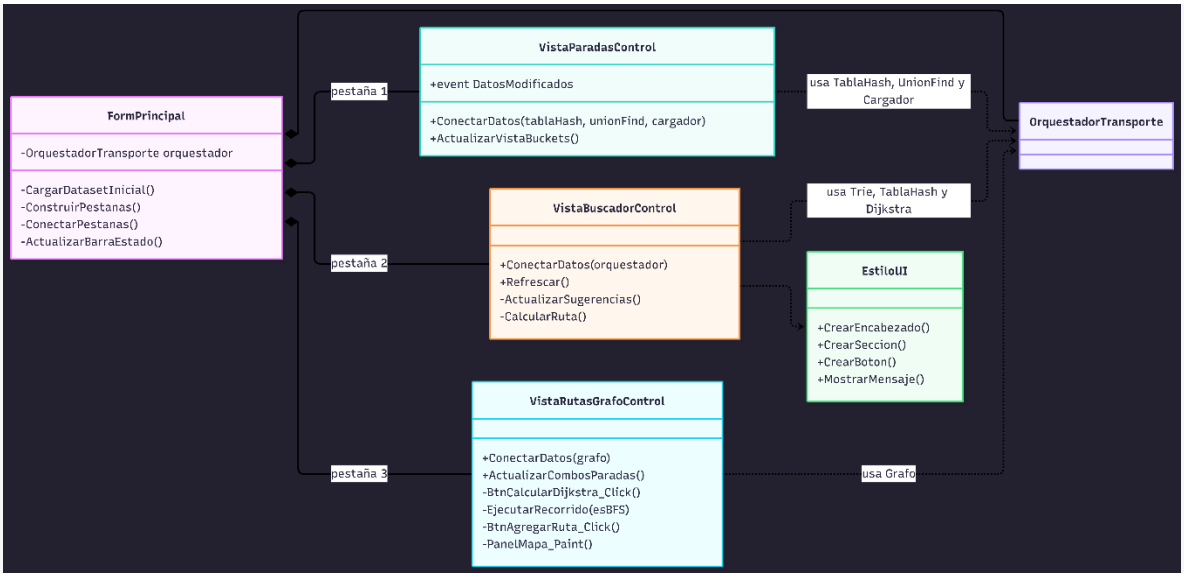
Sincronización de índices. Cada `Parada` guarda su `Indice`. El Grafo numera los vértices 0, 1, 2… en orden de inserción y el Union-Find hace lo mismo, por lo que ambos coinciden; `CargadorCSV.RegistrarParada` verifica esa igualdad y lanza una excepción si se desincronizan. Al eliminar una parada el Grafo **no reindexa**: deja la posición en `null` para que los índices del Union-Find sigan siendo válidos.

4. Diagrama de clases

4.1 Núcleo (`TransporteUrbanoEDD.Core`)

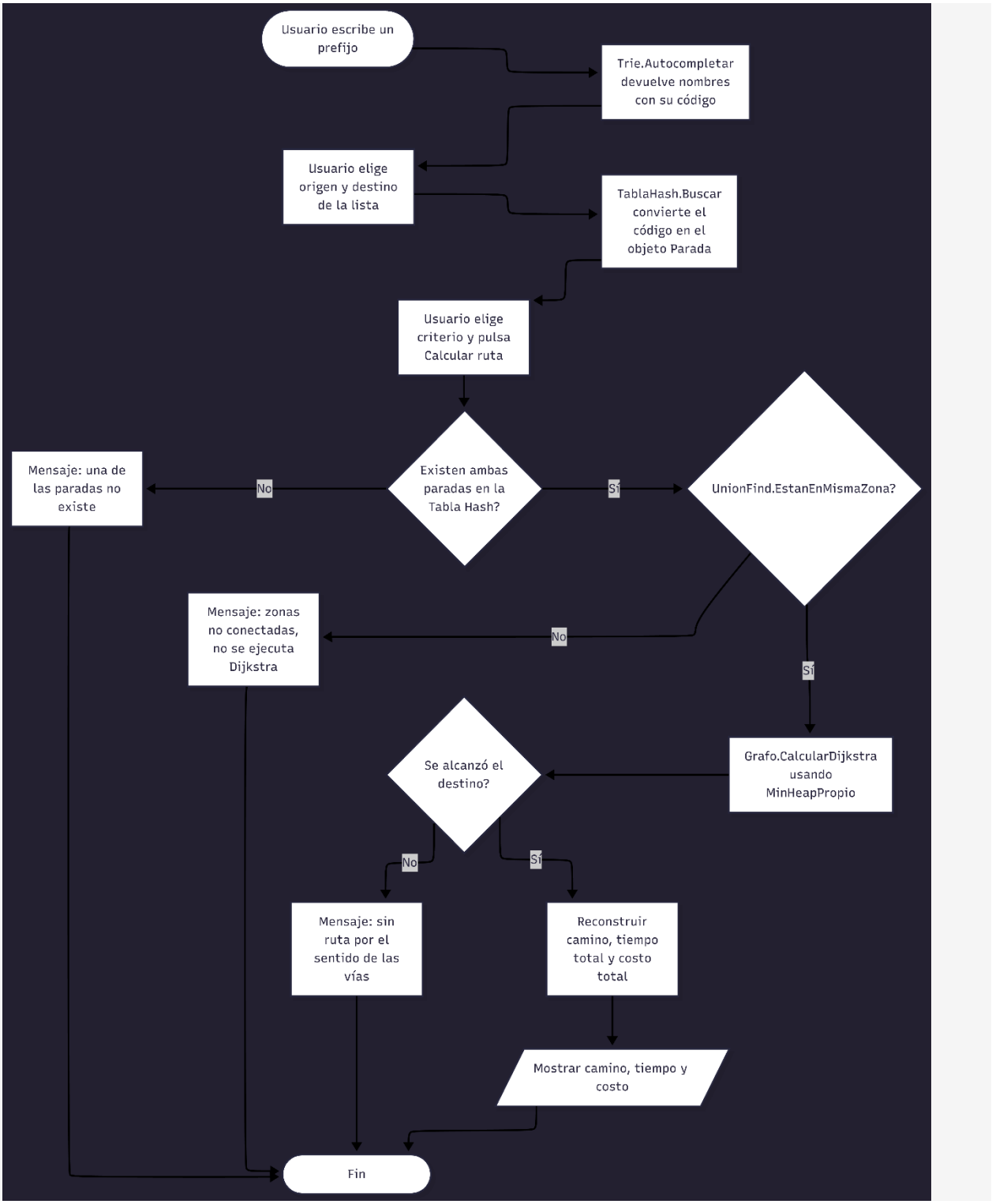


4.2 Interfaz (`TransporteUrbanoEDD.UI`)

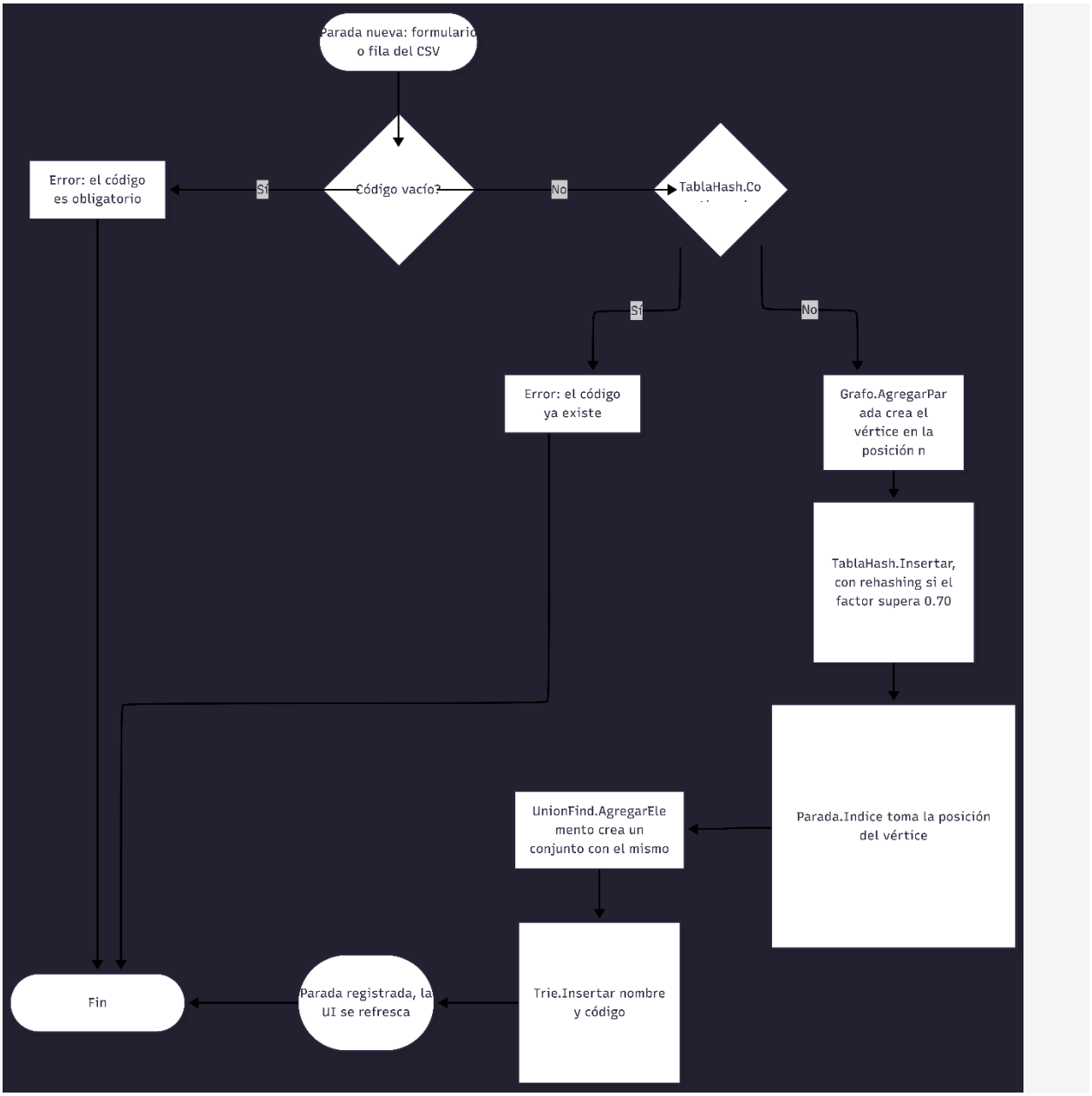


5. Diagramas de flujo

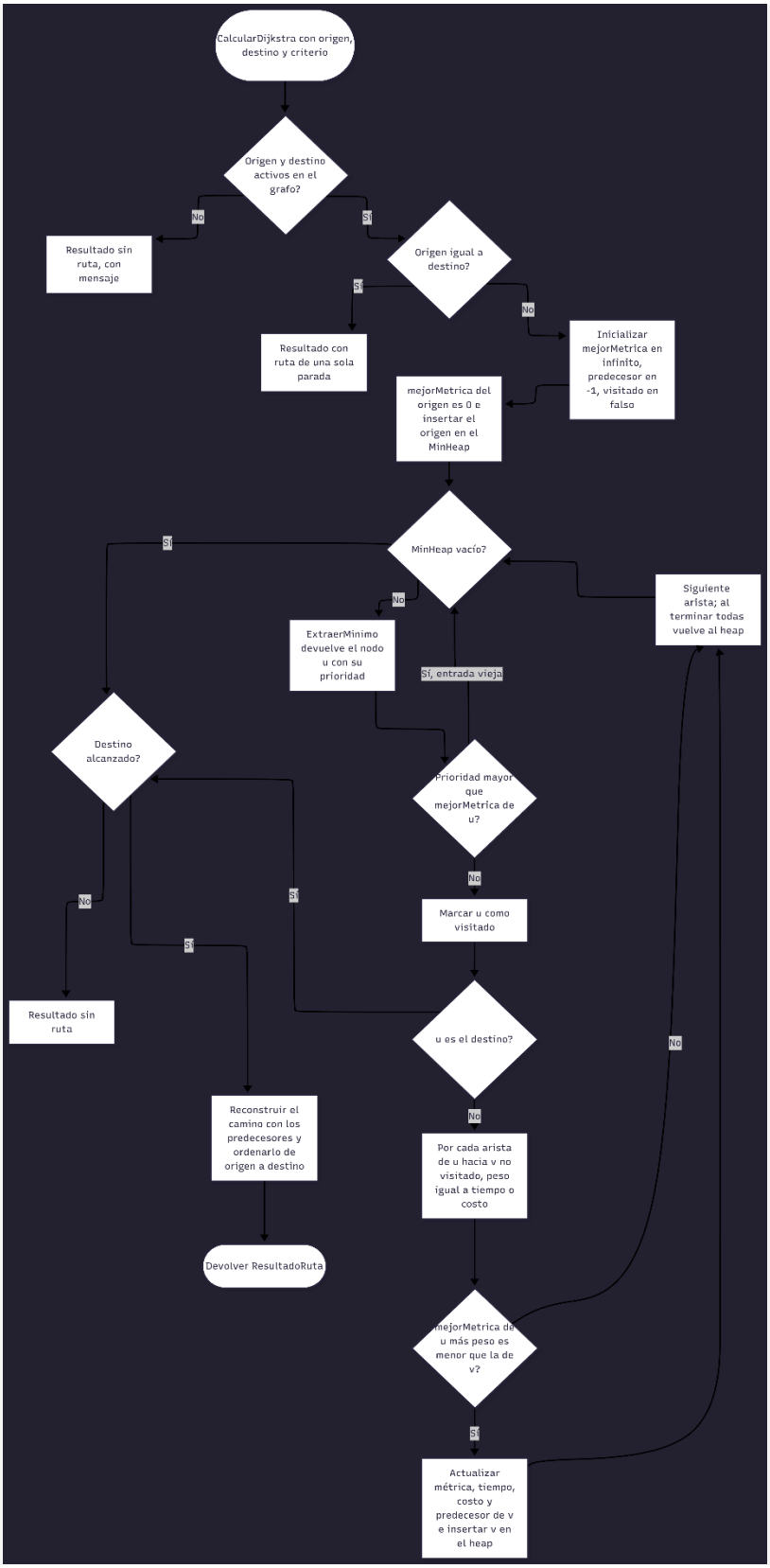
5.1 Calcular una ruta por nombre (flujo integrado de las cuatro estructuras)



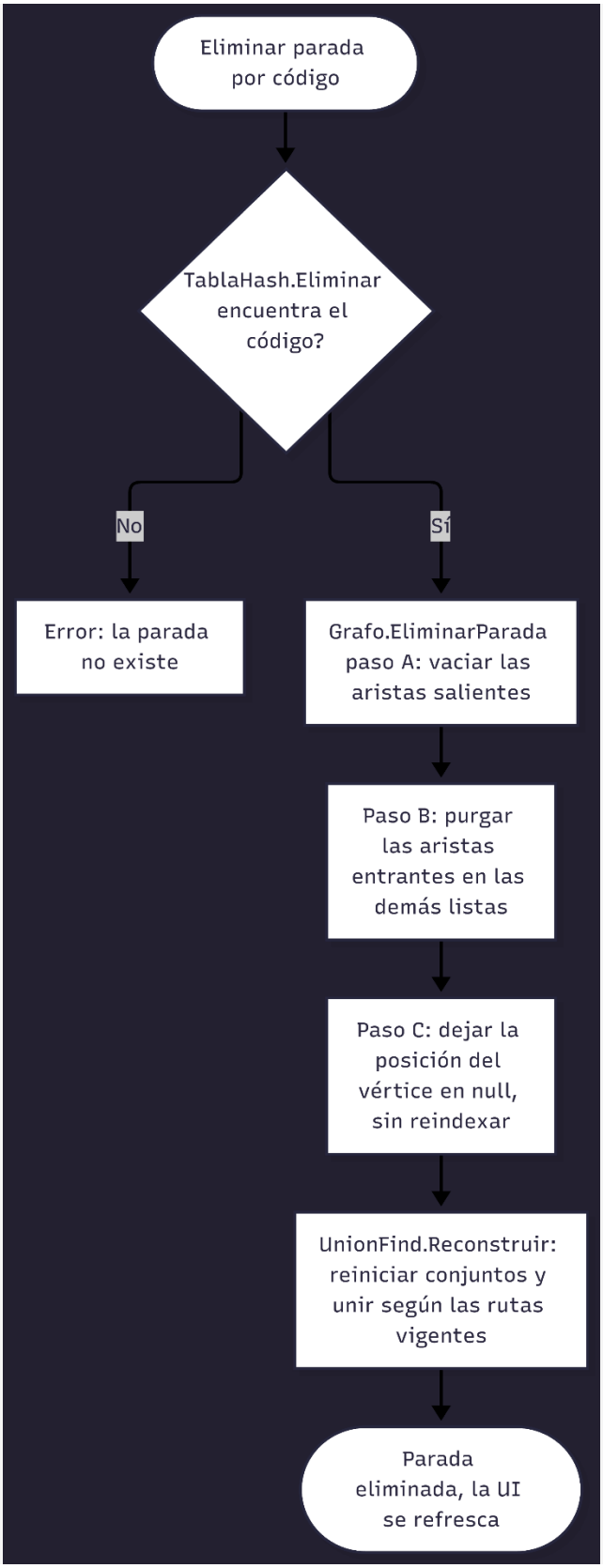
5.2 Registrar una parada



5.3 Algoritmo de Dijkstra (`GrafoTransporte.CalcularDijkstra`)



5.4 Eliminar una parada



6. Explicación de la implementación

6.1 Modelos

- **`Parada`**: código (solo lectura, normalizado a mayúsculas y sin espacios), nombre, coordenadas X/Y para el mapa, zona descriptiva e `Indice`. `Equals` compara por código sin distinguir mayúsculas.
- **`AristaRuta`**: origen, destino, **tiempo** (minutos), **costo** (quetzales) y bandera de bidireccionalidad. Rechaza valores negativos. La doble ponderación permite optimizar por tiempo o por costo con el mismo grafo.
- **`ResultadoRuta`**: ruta encontrada o motivo por el que no existe, resumen textual (`PAR-01 -> PAR-03 -> …`), totales y la secuencia de códigos para pintar el mapa.

6.2 Lista enlazada propia (`ListaPropia<T>`)

Lista simple con referencias a **cabeza y cola**, lo que hace que `Agregar` sea O(1). Implementa `IEnumerable<T>` con `yield return` para poder usar `foreach`. Es la pieza base de todo lo demás: cadenas de la Tabla Hash, listas de adyacencia del Grafo, hijos de cada nodo del Trie, resultados de búsquedas y recorridos.

6.3 Tabla Hash (`TablaHashParadas`)

- **Clave**: el código de la parada (`PAR-01`), normalizado igual que en `Parada`.
- **Función hash** (método de Horner, base 31): `h = (h * 31 + c) % capacidad` por cada carácter. El módulo en cada paso evita desbordamientos y deja el resultado en `0 … capacidad-1`.
- **Colisiones**: encadenamiento separado. Cada *bucket* es una `ListaPropia<Parada>`; si dos códigos caen en el mismo bucket, se encadenan.
- **Rehashing**: capacidad inicial 11. Antes de insertar se proyecta el factor de carga `(n+1)/capacidad`; si supera **0.70**, la capacidad pasa a `2·capacidad + 1` y cada parada se vuelve a distribuir con la nueva capacidad. `Rehashes` cuenta cuántas veces ocurrió.
- **Operaciones**: `Insertar` (rechaza duplicados), `Buscar`, `Contiene`, `Eliminar`, `ObtenerTodas`, `ObtenerBucket` (para la vista de buckets de la UI) y `Mostrar` (impresión en consola).

6.4 Trie (`TrieParadas`)

Árbol de prefijos sobre el **nombre** de la parada (en minúsculas). Cada `NodoTrie` guarda sus hijos en una `ListaPropia<ParHijoTrie>` (carácter + nodo hijo), una marca `EsFinDePalabra` y, en el nodo final, el nombre original y el código de la parada.

- `Insertar(nombre, codigo)`: recorre o crea un nodo por carácter y marca el último.

- `Autocompletar(prefijo)`: desciende por el prefijo y, desde ese nodo, recolecta recursivamente todos los nombres terminados; devuelve textos con el formato `Nombre (CÓDIGO)`.
- `Eliminar(nombre)`: borrado recursivo que quita nodos que quedan sin hijos y sin marca de fin, sin afectar a otras palabras que comparten prefijo.

El Trie no guarda el objeto `Parada`, solo su código: la interfaz obtiene la parada completa preguntándole a la Tabla Hash. Así cada estructura hace una sola tarea.

6.5 Union-Find (`UnionFindZonas`)

Conjuntos disjuntos sobre los índices de las paradas, con dos arreglos: `padre[i]` y `rango[i]`.

- `AgregarElemento`: crea un conjunto unitario (duplica los arreglos si se llenan).
- `Find(x)`: devuelve la raíz con **compresión de caminos**: al volver de la recursión cada nodo queda apuntando directo a la raíz.
- `Union(a, b)`: **unión por rango**: el árbol más bajo se cuelga del más alto. Devuelve `false` si ya estaban en la misma zona.
- `EstanEnMismaZona(codA, codB)`: traduce códigos a índices con la Tabla Hash y compara raíces. Es la validación previa a Dijkstra.
- `Reconstruir`: Union-Find no puede separar conjuntos, así que al eliminar paradas se reinicia y se vuelve a unir según las rutas vigentes. `CantidadZonas` cuenta solo las zonas de paradas activas.

Una *zona* es un componente conexo de la red: se une a las dos paradas de cada ruta sin importar el sentido.

6.6 Grafo (`GrafoTransporte`)

- **Representación**: arreglo `vertices` (paradas) y arreglo `adyacencias` (una `ListaPropia<AristaRuta>` por vértice), ambos con crecimiento por duplicación. Una ruta bidireccional se guarda como **dos aristas dirigidas**; una de un solo sentido, como una.
- **Eliminación** (*baja del vértice + purga total de aristas*): se vacía la lista saliente, se eliminan de las demás listas las aristas cuyo destino sea la parada (manipulando `Cabeza`, `Siguiente` y `Cola`) y la posición queda en `null`. También existe `EliminarRuta`.
- **BFS**: recorrido por niveles con una `Queue<int>` de control y un arreglo `visitados`. Devuelve las paradas en orden de visita.
- **DFS**: recorrido en profundidad recursivo con el mismo arreglo `visitados`.
- **Dijkstra**: usa `MinHeapPropio<int>` como cola de prioridad. Mantiene arreglos `mejorMetrica`, `tiempoAcumulado`, `costoAcumulado`, `predecesor` y `visitado`. El criterio (tiempo o costo) solo cambia el peso que se suma. Las entradas desactualizadas del heap se descartan al extraerlas. Al llegar al destino se detiene y reconstruye el camino con un arreglo primitivo (sin pila nativa).

6.7 Min-Heap propio (`MinHeapPropio<T>`)

Montículo binario mínimo sobre un arreglo de `NodoHeap<T>` (elemento + prioridad). `Insertar` agrega al final y **flota**; `ExtraerMinimo` mueve el último a la raíz y la **hunde**; el arreglo se duplica cuando se llena. Padre de `i`: `(i-1)/2`; hijos: `2i+1` y `2i+2`.

6.8 Carga y guardado de CSV (`CargadorCSV`)

- **Paradas** (`Codigo,Nombre,X,Y,Zona`; la zona es opcional) y **rutas** (`Origen,Destino,Tiempo,Costo,Bidireccional`; acepta `true/false`, `1/0`, `si/no`).
- Se ignoran líneas vacías, comentarios (`#`) y el encabezado. Los decimales usan punto (`InvariantCulture`) y se lee en UTF-8.
- Una línea con error **no detiene la carga**: se omite y se anota en `ResultadoCarga.Errores` con su número de línea; las demás se cargan.
- Solo se aceptan archivos `.csv` o `.txt`. También guarda las paradas y rutas actuales (las rutas bidireccionales se escriben una sola vez).
- Es el único punto por el que se registran paradas y rutas en las cuatro estructuras a la vez (usado por el CSV y por el formulario de la pestaña 1).

6.9 Orquestador (`OrquestadorTransporte`)

Crea las estructuras una sola vez y expone `BuscarRutaOptima(origen, destino, porTiempo)`, que implementa el flujo del enunciado: **(1)** Tabla Hash comprueba que ambas paradas existan; **(2)** Union-Find comprueba que estén en la misma zona; **(3)** solo entonces se ejecuta Dijkstra. Así no se gasta tiempo calculando rutas imposibles y se devuelve un mensaje claro en cada caso.

6.10 Interfaz gráfica

- **Pestaña 1, Paradas y Zonas**: CRUD de paradas, carga/guardado de CSV con lista de errores, verificador de conectividad (Union-Find) y vista de la Tabla Hash bucket por bucket, con las colisiones resaltadas.
- **Pestaña 2, Buscador**: autocompletado con Trie en cada pulsación, detalle desde la Tabla Hash y cálculo de ruta con un panel que muestra paso a paso qué hizo cada estructura.
- **Pestaña 3, Rutas y Mapa**: Dijkstra, BFS, DFS, registro de conexiones viales y un mapa dibujado con GDI+ donde se resaltan origen, destino, ruta óptima y paradas visitadas. Un clic sobre una parada del mapa la selecciona como origen.
- **Barra de estado**: número de paradas, zonas desconectadas, factor de carga y aristas del grafo.

7. Análisis de complejidad (Big-O)

Notación. `n` = paradas (vértices), `m` = rutas (aristas dirigidas), `c` = capacidad de la Tabla Hash, `k` = largo de un código, `L` = largo de un nombre o prefijo, σ = hijos por nodo del Trie, `S` = nodos del subárbol que se recorre al autocompletar, `d` = grado de un vértice, `α(n)` = inversa de Ackermann (menor que 5 para cualquier `n` real).

7.1 Lista enlazada propia

Operación	Complejidad	Observación
`Agregar`	O(1)	Usa la referencia a la cola
`Obtener(i)`	O(n)	Debe caminar desde la cabeza; para recorrer todo se usa `foreach`
`Contiene`, `Eliminar`	O(n)	Búsqueda secuencial
Recorrido con `foreach`	O(n)	

7.2 Tabla Hash

Operación	Promedio	Peor caso	Observación
`FuncionHash`	O(k)	O(k)	`k` es corto y acotado, se trata como constante
`Buscar` / `Contiene`	**O(1)**	O(n)	Peor caso: todos los códigos en un mismo bucket
`Insertar`	**O(1)**	O(n)	Incluye la comprobación de duplicado
`Eliminar`	**O(1)**	O(n)	
Rehashing	O(c + n)	O(c + n)	Ocurre solo al superar 0.70; como la capacidad se duplica, el costo **amortizado** por inserción sigue siendo O(1)
`ObtenerTodas`	O(c + n)	O(c + n)	Recorre todos los buckets
Espacio	O(c + n)		

Con factor de carga ≤ 0.70 y una función hash que reparte bien, la longitud esperada de una cadena es menor que 1, por eso el costo promedio es constante.

7.3 Trie

Operación	Complejidad	Observación
`Insertar`	O(L·σ)	Por cada carácter se busca el hijo en una lista de a lo sumo σ elementos
`Autocompletar`	O(L·σ + S)	Descender por el prefijo y luego recolectar el subárbol (S crece con el número de resultados)
`Eliminar`	O(L·σ)	Borrado recursivo con poda de nodos vacíos
Espacio	O(suma de las longitudes de los nombres)	Los prefijos comunes se comparten

Como σ está acotado por el alfabeto de los nombres (letras, espacios y acentos), el descenso cuesta en la práctica O(L), **independiente de cuántas paradas existan**.

7.4 Union-Find

Operación	Complejidad	Observación
`AgregarElemento`	O(1) amortizado	Duplica el arreglo cuando se llena
`Find`	**O(α(n))** amortizado	Compresión de caminos
`Union`	**O(α(n))** amortizado	Unión por rango
`EstanConectadas`	O(α(n))	Dos `Find`
`EstanEnMismaZona`	O(1) prom. + O(α(n))	Dos búsquedas en la Tabla Hash y dos `Find`
`Reconstruir`	O(n + m·α(n))	Reinicia y une de nuevo según las rutas
Espacio	O(n)	Dos arreglos de enteros

7.5 Grafo (lista de adyacencia)

La columna *Diseño* es la complejidad teórica del algoritmo; la columna *Implementación actual* incluye el costo de `ObtenerIndiceParada`, que busca el código recorriendo el arreglo de vértices (O(n)) cada vez que se llama.

Operación	Diseño	Implementación actual
Espacio	O(n + m)	O(n + m)

Operación	Diseño	Implementación actual
`ObtenerIndiceParada`		O(n)
`AgregarParada`	O(1) amortizado	O(n), comprueba duplicado
`AgregarRuta`	O(1)	O(n), dos búsquedas de índice
`EliminarRuta`	O(d)	O(n + d)
`EliminarParada`	O(n + m)	O(n + m)
`ObtenerTodasLasRutas`	O(m)	O(n·m)
BFS	O(n + m)	O(n·m)
DFS	O(n + m)	O(n·m)
Dijkstra	O((n + m) log n)	O(n·m + m log n)

Con la red de ejemplo (21 paradas y 36 rutas) la diferencia es imperceptible. La mejora está descrita en la sección 8.

7.6 Min-Heap propio

Operación	Complejidad
`Insertar`	O(log n) (amortizado, por el redimensionado)
`ExtraerMinimo`	O(log n)
`VerMinimo`	O(1)
`Limpiar`	O(n)

Dijkstra inserta una entrada nueva cada vez que mejora una distancia (no modifica las existentes), así que el heap llega a tener a lo sumo `m` entradas: O(m log m) = O(m log n).

7.7 Carga de datos y flujo integrado

Operación	Complejidad
`RegistrarParada`	O(n + L·σ): Hash O(1), Grafo O(n), Union-Find O(1), Trie O(L·σ)
`RegistrarRuta`	O(n): dos búsquedas Hash, Grafo O(n), Union-Find O(α(n))
`CargarParadasCSV` (n líneas)	$O(n^2)$ en el peor caso

Operación	Complejidad
`CargarRutasCSV` (m líneas)	O(m·n)
`GuardarParadasCSV`	O(c + n)
`BuscarRutaOptima`	O(1) + O(α(n)) + costo de Dijkstra

Las dos validaciones previas del Orquestador cuestan casi nada comparadas con Dijkstra, y evitan ejecutarlo cuando es seguro que no habrá ruta.

7.8 Comparación con las alternativas nativas de .NET

Necesidad	Estructura propia	Alternativa nativa	Comparación
Localizar parada por código	Tabla Hash: O(1) promedio	`Dictionary<string, Parada>`: O(1) promedio; `List<Parada>.Find`: O(n)	Mismo orden que `Dictionary`. Se implementa propia para controlar colisiones, factor de carga y rehashing, y poder mostrarlos en la UI
Autocompletar por prefijo	Trie: O(L·σ + S)	`List<string>` con `StartsWith`: O(n·L) en cada pulsación; `Dictionary` no sirve para prefijos	El Trie no recorre las paradas que no coinciden
¿Misma zona?	Union-Find: O(α(n))	BFS/DFS por consulta: O(n + m); `HashSet` por zona, que hay que fusionar a mano: O(n)	Consulta casi constante y actualización incremental
Cola de prioridad de Dijkstra	Min-Heap: O(log n)	`PriorityQueue<T, P>`: O(log n); `SortedSet`: O(log n); lista ordenada: O(n)	Mismo orden que `PriorityQueue`
Adyacencia del grafo	Arreglo de listas enlazadas: O(n + m) de espacio	`Dictionary<string, List<Arista>>`; matriz `double[,]`: O(n^2)	La red de transporte es dispersa: la lista de adyacencia ahorra espacio y recorre solo los vecinos reales
Colecciones de resultados	`ListaPropia<T>`: `Agregar` O(1)	`List<T>`: `Add` O(1) amortizado, acceso por índice O(1)	Igual al agregar; el acceso por índice es peor, por eso se recorre con `foreach`

Necesidad	Estructura propia	Alternativa nativa	Comparación
Orden de visita en BFS	`Queue<int>` (nativa, permitida)		Solo como cola de control

7.9 Justificación de la elección de cada estructura

Tabla Hash. La operación más frecuente del sistema es localizar una parada por su código exacto: al registrar una ruta, al validar origen y destino, al traducir un código a su índice en el Union-Find y al mostrar el detalle de una sugerencia. El código es una clave única y no se necesita orden, así que el hash da O(1) promedio frente a O(log n) de un árbol o O(n) de una lista. Se eligió *encadenamiento separado* porque la eliminación es simple (quitar un nodo de la cadena), tolera colisiones sin amontonar posiciones vecinas y permite visualizar cada bucket. La función de Horner usa todos los caracteres y su posición (así `PAR-01` y `PAR-10` se reparten distinto) y el límite de 0.70 mantiene las cadenas cortas.

Trie. El autocompletado es una búsqueda por *prefijo*, algo que una tabla hash no puede hacer (solo compara igualdad exacta). Un arreglo ordenado exigiría búsqueda binaria más recorrer el rango, y una lista simple, revisar todas las paradas en cada tecla. El Trie cuesta según el largo del prefijo, comparte prefijos comunes («Parque Central», «Parque Benito Juarez») y entrega los resultados al llegar al nodo. Los hijos se guardan en listas enlazadas para no fijar un alfabeto (admite espacios, tildes y ñ).

Union-Find. Antes de cada Dijkstra hay que saber si dos paradas están en la misma zona, y la red solo *crece* por uniones (cada ruta nueva une dos paradas). Es el caso ideal: cada consulta es casi O(1) gracias a la compresión de caminos y la unión por rango. Repetir un BFS en cada consulta costaría O(n + m). Su debilidad es que no puede deshacer uniones, y por eso se reconstruye al eliminar paradas.

Grafo con listas de adyacencia. Cada parada se conecta con pocas vecinas, así que la red es dispersa: una matriz gastaría $O(n^2)$ de memoria y recorrer los vecinos costaría O(n). La arista guarda tiempo y costo para optimizar por cualquiera de los dos criterios. Las vías de un solo sentido son aristas dirigidas y las bidireccionales son dos aristas. Eliminar sin reindexar mantiene estables los índices compartidos con el Union-Find.

Min-Heap. Dijkstra extrae el vértice de menor distancia una y otra vez. Con un heap cada extracción es O(log n); buscando el mínimo linealmente el algoritmo costaría $O(n^2)$.

Lista enlazada. Es la base de cadenas, adyacencias, hijos y resultados porque se agrega al final en O(1) y casi nunca se necesita acceso por posición.